
LaFree World Model: Label-Efficient Latent Action Inference for JEPA World Models

Ciem Cornelissen Sam Leroux Pieter Simoens

IDLab, Department of Information Technology

Ghent University - imec, Belgium

{ciem.cornelissen, sam.leroux, pieter.simoens}@ugent.be

Abstract

JEPA-style world models learn latent dynamics from pixels, but typically require action labels for every training trajectory. This limits their use of large interaction datasets whose observations are available without synchronized actions. We present LaFreeWM, which replaces labeled action conditioning during world-model pre-training with discrete latent actions inferred from consecutive encoded observations. Vector quantization restricts how much transition-specific detail can pass through each inferred action, while a latent-action decoder, a JEPA transition objective, and end-to-end optimization shape the representation for downstream control. A small labeled subset then grounds the pretrained predictor in raw actions for planning. Across four benchmarks and both 1% and 10% label budgets, LaFreeWM outperforms label-matched LeWM in every environment and Garrido et al. (Noisy) in three of four; at 10%, it also surpasses fully supervised PLDM in all four environments and fully supervised LeWM in three. Its compact native planner completes the matched workload in 0.287 seconds per solve, $2.44\times$ faster than Garrido et al. (Noisy). Finally, removing quantization reduces held-out prediction MSE by about $60\times$ yet lowers long-horizon planning success from 60.7% to 18.0%, exposing a prediction-planning mismatch in which better observational prediction does not imply better control.

1 Introduction

Learning world models from raw sensory observations is a central challenge in building general-purpose agents. A world model that captures how environments evolve under different actions can support planning, policy optimization, and counterfactual reasoning, all without requiring task-specific reward signals during training [Ha and Schmidhuber, 2018, LeCun, 2022]. Reconstruction-based approaches such as the Dreamer family [Hafner et al., 2020, 2023] learn latent dynamics models for policy optimization via imagination, but require pixel-level decoding. Joint Embedding Predictive Architectures (JEPAs) provide an attractive alternative: encode observations into compact latent states and predict future embeddings conditioned on actions, avoiding generative complexity [LeCun, 2022].

Recent JEPA-based world models such as LeWorldModel (LeWM) [Maes et al., 2026] have demonstrated that stable end-to-end latent world models can be trained from pixels using remarkably simple objectives (a next-embedding prediction loss plus SIGReg, a single anti-collapse regularizer introduced in LeJEPA [Balestriero and LeCun, 2025]), while achieving fast latent planning competitive with foundation-model-based alternatives like DINO-WM [Zhou et al., 2024]. However, all these methods treat the action interface as a supervised input: ground-truth action labels must be available on every training trajectory. This requirement limits scalability to large, cheaply collected video datasets where action labels are absent, partial, or expensive to obtain.

The label dependency is not merely inconvenient; it is a structural bottleneck. In robotics, action labels come from proprioceptive sensors or teleoperation logs that must be synchronized with camera feeds; in video datasets harvested from the web, no action labels exist at all. Even in simulation, relabeling trajectories under different action conventions requires re-collection. A world model that could learn its dynamics from observation sequences alone, and only require a small number of action labels at deployment time, would be substantially more scalable.

A natural solution is to infer actions from data. Inverse dynamics models (IDMs) that predict actions from state-transition pairs have been explored in self-supervised exploration [Pathak et al., 2017] and video-based latent action discovery [Bruce et al., 2024], but integrating them into planning-capable world models introduces a specific failure mode: the *prediction-loss paradox*. When the IDM’s output conditions the prediction pathway and the prediction gradient flows back through the IDM unconstrained, the latent codes are reshaped to encode state-transition detail rather than action structure, achieving low prediction loss but producing codes that are useless for planning. More broadly, this is a shortcut-learning problem: predictive objectives can be satisfied by representations that preserve the wrong information for downstream control unless their geometry and information content are constrained.

In this work, we study how to integrate a self-supervised IDM into a JEPA world model while maintaining action codes that support effective planning. Our approach, LaFreeWM, extends LeWM by inferring actions from pairs of consecutive encoded observations and quantizing them before they condition the dynamics predictor. A latent-action decoder and a JEPA transition objective shape these discrete actions, and gradients from the prediction objective pass through the quantized action path so that the representation and dynamics model are learned jointly. Once pretraining is complete, limited labeled data replaces the inferred-action path with direct raw-action conditioning, allowing the same predictor to support planning with only 1% or 10% of the trajectories labeled.

Contributions.

- We present LaFreeWM, an end-to-end JEPA dynamics predictor trained without action labels by conditioning on observation-inferred discrete actions, then grounded with limited labels for raw-action planning.
- We isolate a target-information shortcut specific to this coupled IDM–predictor interface: the training-time IDM sees the future state that the deployed planner must predict. A direct prediction-versus-planning diagnostic shows why low observational prediction error alone is insufficient for counterfactual control.
- We conduct a complete controlled single-step study that retrains and evaluates LeWM, PLDM, DINO-WM, and a common-interface adaptation of Garrido et al. [Garrido et al., 2026] under the same task and evaluation protocol, together with supervision-matched controls and fixed-interface ablations.

2 Related Work

JEPA-style world models. Joint Embedding Predictive Architectures [LeCun, 2022] learn latent representations by predicting future embeddings rather than reconstructing pixels. Self-supervised variants such as I-JEPA [Assran et al., 2023] and V-JEPA [Bardes et al., 2024] use masked prediction with EMA target encoders. For latent-dynamics planning from offline trajectories, PLDM [Sobal et al., 2025] trains a JEPA-style latent dynamics model with variance-covariance regularization, temporal alignment, and inverse-dynamics losses. DINO-WM [Zhou et al., 2024] sidesteps collapse by freezing a pretrained DINOv2 [Oquab et al., 2024] encoder and learning dynamics over token-level visual features, achieving strong performance at the cost of end-to-end adaptation and substantially higher planning cost. LeWM [Maes et al., 2026] demonstrates that a two-term objective (next-embedding MSE plus SIGReg anti-collapse regularization) suffices for stable end-to-end JEPA training from pixels with a compact 15M-parameter model. V-JEPA 2 [Assran et al., 2025] instead action-free-pretrains a video representation before post-training a frozen-encoder, action-conditioned world model for planning, whereas VLA-JEPA [Sun et al., 2026] uses future latents only as supervision targets and deploys an action-prediction policy. Concurrent Delta-JEPA [Zhang et al., 2026b] is an end-to-end JEPA planner whose latent-difference decoder reconstructs executed actions, and therefore uses action supervision to shape transition geometry. Within this landscape, our focus is an

observation-inferred action interface inside the end-to-end dynamics predictor, later grounded for raw-action CEM.

Inverse dynamics and latent action models. Inverse dynamics models (IDMs) that infer actions from state transitions have been used for self-supervised exploration [Pathak et al., 2017], representation learning [Agrawal et al., 2016], and video-based latent action discovery. Genie [Bruce et al., 2024] learns a VQ-based latent action model from video, using a discrete bottleneck to discover controllable action modes without labels. LAPA [Ye et al., 2024] similarly learns discrete pseudo-actions for VLA pretraining before limited-label robot-action fine-tuning, while LAWM [Tharwat et al., 2025] learns latent actions through world modeling to pretrain imitation models. CLAM [Liang et al., 2025] uses continuous latent actions and jointly trains an action decoder to ease limited-label grounding. MVP-LAM [Lee et al., 2026] uses synchronized cross-viewpoint reconstruction to suppress viewpoint-specific shortcuts, and CLAP [Zhang et al., 2026a] contrastively aligns human-video transitions with quantized robot-action latents. These methods suggest complementary ways to enrich our single-view objective with viewpoint consistency and cross-domain alignment.

At larger generative-model scale, DreamDojo [Gao et al., 2026] pretrains continuous proxy actions from human video before robot post-training, DreamGen [Jang et al., 2025] recovers pseudo-actions from world-model-generated robot trajectories for policy training, and DreamZero [Ye et al., 2026] jointly models video and robot actions for closed-loop control. Together, these systems demonstrate the potential of latent-action learning at scale; our work advances this direction by integrating observation-inferred actions into an end-to-end JEPA world model and grounding the predictor for efficient raw-action planning.

The closest planning formulation is Learning Latent Action World Models in the Wild [Garrido et al., 2026], which combines frozen V-JEPA representations, constrained continuous latent actions learned from diverse videos, and a controller from known actions to the learned latent interface. We implement Garrido et al. (Noisy) as a controlled adaptation with the same compact simulation and planning interface used in our experiments. Because latent-action VLA policies, generative video models, and CEM dynamics models differ in outputs, supervision, and deployment, we separate conceptual comparisons from common-interface experiments.

Information bottlenecks in action representations. At pretraining, our IDM sees $(\mathbf{z}_t, \mathbf{z}_{t+1})$, including the target that the dynamics model predicts; at deployment, CEM must score a proposed action without access to $\mathbf{z}_{t+1}$. An unconstrained conditioning latent can therefore transmit future-transition detail and reduce prediction error without becoming an executable action interface. This target-information shortcut connects our setting to the broader visual-entanglement problem studied above. Stop-gradient can block predictor–target coupling [Grill et al., 2020], while VAE and VQ-VAE bottlenecks constrain capacity [Kingma and Welling, 2013, van den Oord et al., 2017]. Our fixed-codebook VQ interface bounds the sample-specific channel; matched ablations then test whether that constraint and the LAM objective improve planning in the compact JEPA/CEM setting.

3 Method

We describe LaFreeWM, a JEPA-style world model that replaces ground-truth action conditioning with self-supervised latent action codes inferred by an inverse dynamics model (IDM). The model builds on the LeWM framework [Maes et al., 2026]: an end-to-end ViT encoder maps pixel observations to compact CLS latent states, and an inline predictor models dynamics in latent space conditioned on actions. LaFreeWM replaces the action interface with a VQ-bottlenecked IDM pathway, enabling world-model training without action labels.

3.1 Architecture overview

LaFreeWM uses a Vision Transformer (ViT-Tiny: 12 layers, 3 heads, 192-dim embeddings, patch size 14) as its backbone. The encoder produces CLS embeddings from pixel observations using a standard HuggingFace ViT. The prediction pathway uses inline CLS prediction tokens that pass through a LeWM-style predictor (6 layers, 16 heads, 2048 MLP dim, 10% dropout, causal masking) with AdaLN action conditioning.

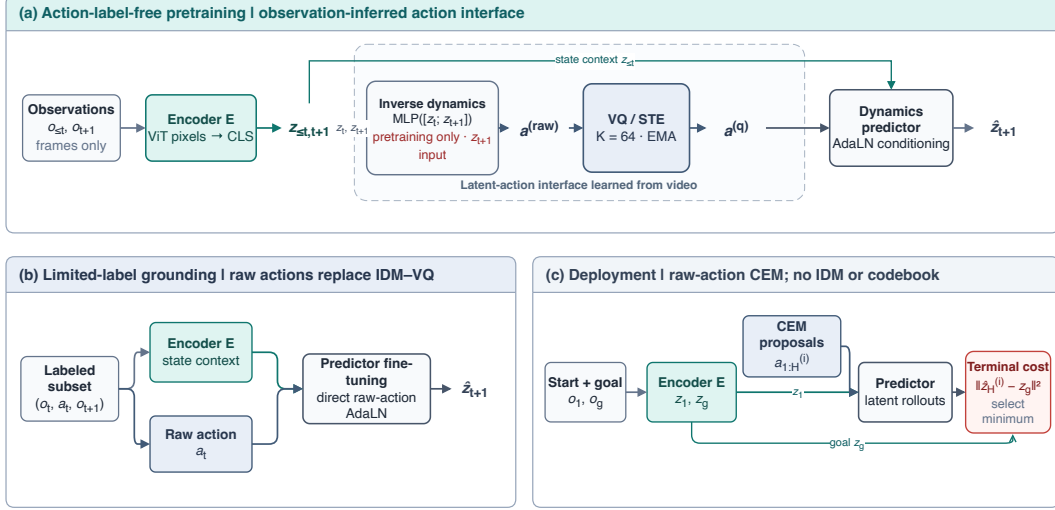


Figure 1: The training and deployment interfaces of LaFreeWM. **(a)** During action-label-free pretraining, the IDM observes consecutive encoded states and supplies a quantized action to the dynamics predictor. **(b)** Limited-label grounding replaces the IDM–VQ interface with direct raw-action conditioning. **(c)** At deployment, CEM proposes raw-action sequences and ranks their predicted terminal states; the future observation and IDM are unavailable.

The key architectural change relative to LeWM is the action source. Instead of encoding ground-truth actions through an action-chunk encoder, LaFreeWM infers discrete latent action codes from consecutive CLS embeddings via an IDM head, passes them through a VQ bottleneck, and uses the resulting quantized codes for prediction conditioning. Figure 1 makes the resulting interface change explicit across the three stages of the method. The future state is available to the IDM only during action-label-free pretraining; limited-label grounding then replaces the IDM–VQ path with raw-action conditioning, so deployment can score proposed actions without access to the future observation.

3.2 VQ-bottlenecked inverse dynamics model

The IDM is the core component of LaFreeWM. It infers discrete action codes from observed state transitions, providing action conditioning to the dynamics predictor without ground-truth action labels.

Inverse dynamics head. Given CLS embeddings $\mathbf{z}_t$ and $\mathbf{z}_{t+1}$ from two consecutive time steps, the IDM head computes a continuous action latent:

$$\mathbf{a}^{(\text{raw})} = \tanh(\text{MLP}([\mathbf{z}_t; \mathbf{z}_{t+1}])) \in \mathbb{R}^{d_a}, \quad (1)$$

where $[\cdot; \cdot]$ denotes concatenation and $d_a = 24$ by default. The MLP consists of LayerNorm, Linear, GELU, Linear, and Tanh layers.

VQ bottleneck. The continuous latent passes through a vector-quantization bottleneck with K learned entries:

$$\mathbf{a}^{(q)} = \mathbf{a}^{(\text{raw})} + \text{sg} \left[\mathbf{e}_k - \mathbf{a}^{(\text{raw})} \right], \quad k = \arg \min_j \|\mathbf{a}^{(\text{raw})} - \mathbf{e}_j\|_2, \quad (2)$$

where sg is stop-gradient and the reported Full model updates codebook entries $\mathbf{e}_j$ by exponential moving averages (EMA) of assigned latents. The straight-through estimator gives $\partial \mathbf{a}^{(q)} / \partial \mathbf{a}^{(\text{raw})} = I$, allowing prediction gradients to reach the IDM, while the predictor receives only one of K shared prototypes. For a fixed trained codebook and transition T , the sample-specific channel obeys

$$I(T; \mathbf{a}^{(q)} | \mathbf{z}_t) \leq H(k | \mathbf{z}_t) \leq H(k) \leq \log_2 K. \quad (3)$$

Equation 3 limits only how much information a single discrete index can transmit through this interface; whether the learned indices capture executable actions is determined by the training objectives and evaluated through planning.

JEPA transition shaping. A self-supervised JEPA loss shapes the action latent to predict state transitions. The transition target is the L2-normalized CLS delta:

$$\mathbf{t}_{t \rightarrow t+1} = \frac{\mathbf{z}_{t+1}^{(\text{sg})} - \mathbf{z}_t^{(\text{sg})}}{\|\mathbf{z}_{t+1}^{(\text{sg})} - \mathbf{z}_t^{(\text{sg})}\|_2}, \quad (4)$$

where $\text{sg}[\cdot]$ denotes stop-gradient, and the JEPA loss is the cosine distance between the predicted transition $\hat{\mathbf{t}} = f_\phi(\mathbf{a}^{(q)})$, where f_ϕ is a small predictor MLP, and this target:

$$\mathcal{L}_{\text{JEPA}} = 1 - \cos(\hat{\mathbf{t}}, \mathbf{t}_{t \rightarrow t+1}). \quad (5)$$

This provides a self-supervised gradient to the IDM that is independent of action labels: the latent code must capture transition structure, not just prediction-convenient information.

Latent action model (LAM) decoder. A dedicated latent action transition decoder predicts the next CLS token from the current state context and the quantized action code:

$$\hat{\mathbf{z}}_{t+1}^{(\text{LAM})} = \text{LAM}(\text{context}(\mathbf{z}_{\leq t}), \mathbf{a}^{(q)}), \quad (6)$$

trained with a detached target,

$$\mathcal{L}_{\text{LAM}} = \left\| \hat{\mathbf{z}}_{t+1}^{(\text{LAM})} - \text{sg}[\mathbf{z}_{t+1}] \right\|_2^2. \quad (7)$$

This decoder has its own parameters and does not share weights with the main predictor. It provides a reconstruction-driven gradient to the decoder, the shared encoder context, and the IDM action path through the straight-through estimator. The codebook entries themselves remain gradient-free and are updated only by EMA assignments.

Joint gradient flow (without detachment). In the main LaFreeWM variant, the IDM’s output is not detached from the main prediction loss. This means the prediction pathway’s gradient reaches the IDM head through the straight-through estimator. The main prediction target is also non-detached, so $\mathcal{L}_{\text{pred}}$ reaches the shared encoder through the action/STE, state-context, and target branches. In contrast, the JEPA and LAM targets use stop-gradient. The VQ bottleneck constrains this interaction to codebook assignments, while the JEPA and LAM losses provide additional shaping signals. Together, these paths let the dynamics predictor and inferred-action representation co-adapt while keeping the action signal restricted to shared discrete prototypes.

3.3 Training objective

The training objective combines the standard JEPA prediction loss with IDM-specific terms:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{pred}}}_{\text{next-CLS MSE}} + \lambda_{\text{sigreg}} \underbrace{\mathcal{L}_{\text{SIGReg}}}_{\text{anti-collapse}} + \lambda_{\text{JEPA}} \underbrace{\mathcal{L}_{\text{JEPA}}}_{\text{IDM transition}} + \lambda_{\text{LAM}} \underbrace{\mathcal{L}_{\text{LAM}}}_{\text{action-model decoder}} + \lambda_{\text{VQ}} \underbrace{\mathcal{L}_{\text{commit}}}_{\text{VQ commitment}}. \quad (8)$$

Default loss weights are: $\lambda_{\text{sigreg}} = 0.09$, $\lambda_{\text{JEPA}} = 0.25$, $\lambda_{\text{LAM}} = 1.0$, $\lambda_{\text{VQ}} = 0.25$. SIGReg is the LeJEPA anti-collapse regularizer applied to encoder CLS representations; it matches their empirical characteristic function to an isotropic Gaussian reference and does not introduce an EMA target encoder [Balestriero and LeCun, 2025]. Figure 2 summarizes where each objective attaches: the main prediction loss follows the joint encoder–IDM path, JEPA and LAM use detached transition targets, and the codebook itself is updated by EMA rather than gradients.

3.4 Planning and deployment interface

At evaluation time, CEM [Rubinstein, 1999] optimizes raw action sequences against a terminal latent cost. Given initial observation o_1 and goal o_g , the planner selects the action sequence that minimizes $\|\hat{\mathbf{z}}_H - \mathbf{z}_g\|_2^2$ after those actions have been mapped into the model’s prediction pathway.

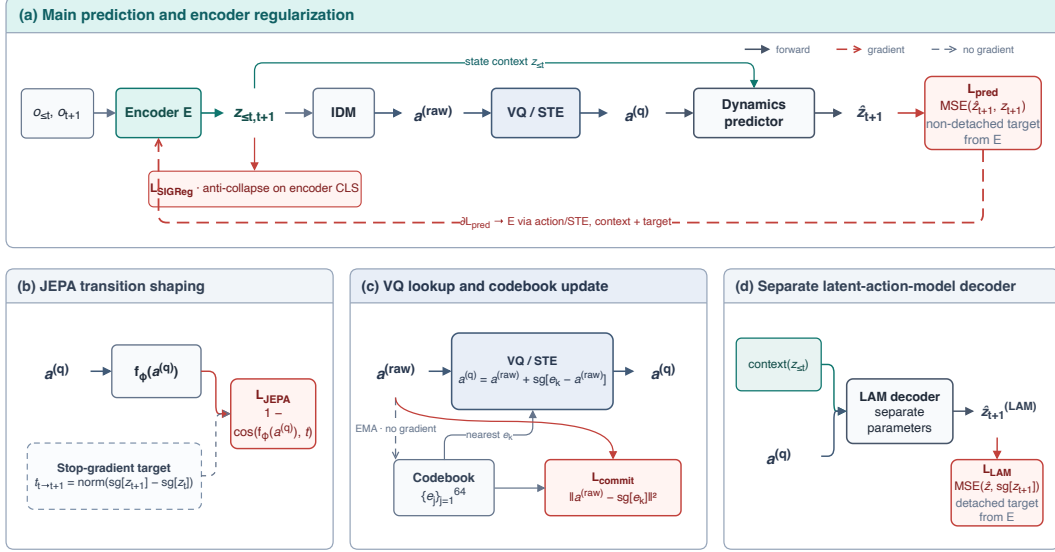


Figure 2: Loss and update paths during action-label-free pretraining. **(a)** The main prediction loss reaches the shared encoder through the action/STE, state-context, and non-detached target branches; SIGReg regularizes encoder CLS states. **(b)** JEPA transition shaping uses a stop-gradient target. **(c)** VQ uses an encoder commitment loss and a non-gradient EMA codebook update. **(d)** The separate LAM decoder uses a detached next-state target.

Because the world model is pre-trained with discrete latent action codes, it cannot directly accept raw actions from the planner. To bridge this gap, we fine-tune the model on a small labeled subset (1% or 10% of the training data with action labels), replacing the VQ-bottlenecked IDM pathway with direct raw-action conditioning. After fine-tuning, CEM proposes raw actions and the predictor evaluates their cost directly, without routing them through the IDM or codebook. The “Labels” column in our results tables indicates the fraction of labeled data used for this fine-tuning step. Thus, action labels are used only for the limited grounding stage rather than for world-model pretraining.

4 Experimental Setup

4.1 Environments

We evaluate on four continuous-control planning benchmarks from the LeWM evaluation suite: **PushT** [Chi et al., 2023] (2D manipulation, history size 3), **TwoRoom** (grid-world navigation, history size 1), **Reacher** (DMC [Tassa et al., 2018] continuous control, history size 1), and **Cube** (OGBench [Park et al., 2024] object manipulation, history size 3). All environments use offline trajectories of pixel observations (96×96 RGB) and continuous actions collected from pseudo-expert policies, loaded via LeWM-format HDF5 datasets. Figure 3 shows example observations from each environment.

4.2 Evaluation protocol

The controlled comparison uses a dataset frameskip of 1 with single-step actions. Methods use the same task trajectories, frameskip, and evaluation implementation; the matched component study additionally fixes the labeled manifest, optimization schedule, raw-action interface, CEM settings, and evaluation seeds. For goal horizons $h \in \{1, 5, 10, 15\}$, the implementation uses evaluation budgets $\{5, 10, 20, 30\}$ primitive steps, respectively. Each cell contains 50 episodes. Values at $h = 15$ report mean and population standard deviation over planner/environment seeds 0, 1, and 2; shorter horizons are seed-0 point estimates.

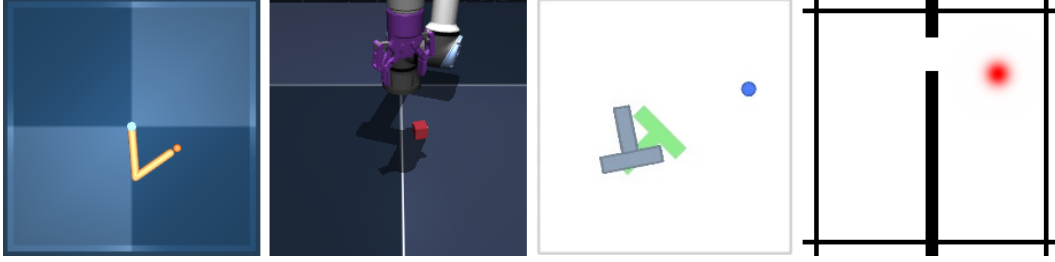


Figure 3: Example observations from the four evaluation environments, ordered left to right: Reacher (DMC continuous control), Cube (OGBench object manipulation), PushT (2D manipulation), and TwoRoom (grid-world navigation).

4.3 Baselines

Our direct supervision-matched baseline is LeWM [Maes et al., 2026], trained using exactly the same 1% or 10% action-labeled episode subset and the same raw-action planning interface. This comparison tests whether action-label-free pretraining improves over learning only from the available labeled trajectories.

We also adapt the noisy continuous latent-action formulation of Garrido et al. [Garrido et al., 2026] to the common simulation and planning interface used by LaFreeWM. It combines a frozen V-JEPA 2-L encoder with a jointly trained continuous IDM and predictor, followed by a known-action-to-latent controller trained on the labeled subset.

Fully supervised LeWM, PLDM [Sobal et al., 2025], and DINO-WM [Zhou et al., 2024] provide additional context. PLDM combines latent prediction with variance–covariance regularization, temporal alignment, and inverse-dynamics supervision, whereas DINO-WM plans over token-level features from a frozen DINOv2 encoder. DINO-WM serves as a heavyweight performance reference: under the matched native workload in Appendix C.4, it requires 42.627 seconds per solve compared with 0.287 seconds for LaFreeWM ($148.6\times$ slower).

4.4 Training details

All component variants use the same optimizer, data, pretraining budget, and limited-label grounding protocol. Complete optimization settings, training budgets, hardware, and codebook configurations are reported in Appendix A; the full sensitivity analysis is in Appendix C.2.

4.5 Evaluation metrics

The primary metric is planning success rate (SR, %): the fraction of episodes where the planner reaches the goal within the eval budget. We report SR at multiple goal horizons for the single-step regime. Secondary metrics include prediction MSE, VQ codebook perplexity, JEPA loss, and fine-tuning loss for IDM diagnostics.

5 Results

5.1 Single-step: data-matched cross-environment comparison

Table 1 separates supervision-matched controls from fully supervised context under the single-step protocol.

Table 1: Single-step planning success (% , $\uparrow$) at $h = 15$. “Full” baselines use all action labels. At 1%/10%, LeWM trains only on the labeled subset, whereas LaFreeWM and Garrido et al. (Noisy) pretrain on trajectories without action annotations before limited-label grounding. LaFreeWM uses a per-environment configuration; fixed-setting results are in Appendix C.2.

Method	Labels	PushT	TwoRoom	Reacher	Cube
LeWM	Full	30.0 ± 10.7	92.7 ± 2.5	62.7 ± 8.2	77.3 ± 3.4
DINO-WM	Full	86.7 ± 3.8	100.0 ± 0.0	61.3 ± 6.2	74.7 ± 6.2
PLDM	Full	20.7 ± 4.1	68.7 ± 9.0	24.0 ± 10.7	68.7 ± 2.5
LeWM	1%	6.7 ± 0.9	50.7 ± 5.2	20.0 ± 4.3	63.3 ± 7.7
Garrido et al. (Noisy)	1%	13.3 ± 3.4	91.3 ± 5.7	6.7 ± 5.2	60.0 ± 9.9
LaFreeWM	1%	33.3 ± 1.9	76.7 ± 10.4	67.3 ± 7.4	70.7 ± 3.4
LeWM	10%	26.0 ± 5.7	78.0 ± 4.9	60.7 ± 6.2	76.7 ± 2.5
Garrido et al. (Noisy)	10%	16.0 ± 8.2	92.7 ± 5.0	61.3 ± 1.9	71.3 ± 2.5
LaFreeWM	10%	60.7 ± 3.4	83.3 ± 9.6	70.7 ± 5.7	82.0 ± 7.1

At both label budgets, LaFreeWM improves over label-only LeWM in all four environments, showing the benefit of action-label-free pretraining when only a fraction of the trajectories carry action labels. Against Garrido et al. (Noisy), LaFreeWM is higher on PushT, Reacher, and Cube, while the continuous baseline is higher on TwoRoom at both budgets. This consistent task-dependent pattern highlights both the gains of the discrete latent-action interface and the remaining challenge on TwoRoom.

Under the matched native planning workload, LaFreeWM requires 0.287 ± 0.008 s per solve, compared with 0.700 ± 0.012 s for Garrido et al. (Noisy), 2.885 ± 0.086 s for LeWM, 2.996 ± 0.017 s for PLDM, and 42.627 ± 0.164 s for DINO-WM. Appendix C.4 reports the full timing protocol and unified-adaptor control.

5.2 Matched IDM ablations

Table 2 holds the PushT data, label-free pretraining budget, 10% labeled manifest, staged raw-action fine-tuning interface, planner settings, and evaluation seeds fixed across both the component and codebook studies.

Table 2: Matched PushT ablations under a common training and deployment interface: planning success (% , $\uparrow$).

Group	Variant	$h = 1$	$h = 5$	$h = 10$	$h = 15$
Label construction	Constant action	92	74	16	26.7 ± 4.1
Component	Without VQ	92	64	26	18.0 ± 3.3
	Without LAM	96	80	20	26.7 ± 1.9
	Separate IDM	98	76	44	40.7 ± 5.2
	Gradient-updated VQ	98	88	60	47.3 ± 5.2
	Detached prediction gradient	100	90	64	56.0 ± 5.9
	Without JEPA loss	100	98	70	58.0 ± 4.3
Codebook	Full, $K = 8$	98	94	68	48.7 ± 5.2
	Full, $K = 16$	100	96	78	60.7 ± 3.4
	Full, $K = 64$	100	96	70	59.3 ± 8.2
	Full, $K = 128$	96	94	56	51.3 ± 3.4

The reference Full model obtains the highest mean $h = 15$ success among the component-ablation variants, and removing or altering any of the coupled pathways reduces long-horizon performance. In the independent pretraining-seed study (Appendix C.3), Full also exceeds No-VQ and No-LAM for every paired seed and exceeds Detached for two of three seeds. Together, these results support the complete objective, including JEPA transition shaping and the joint gradient path, as a coupled design.

The prediction diagnostic exposes why observational fit is insufficient here. Across two independently pretrained PushT replicas, No-VQ achieves held-out feature-prediction MSE 1.595×10^{-4} versus 9.588×10^{-3} for the matched quantized model, about $60\times$ lower, but its $h = 15$ raw-action planning success is 18.0% versus 60.7%. Thus the continuous IDM is a better observational predictor yet a worse counterfactual planner in this setting. Full and No-LAM reach similarly high codebook utilization despite very different planning success, showing that utilization alone does not establish executable action semantics. Appendix C.3 reports conditional pretraining-seed stability, and Appendix Table 3 gives the complete horizon profile.

6 Discussion and Limitations

Label efficiency. The supervision-matched comparison shows that observation-only trajectories provide useful pretraining signal beyond what LeWM can learn from the labeled subset alone. At both the 1% and 10% label budgets, LaFreeWM improves over label-only LeWM in every environment. It also exceeds Garrido et al. (Noisy) on PushT, Reacher, and Cube at both budgets, with TwoRoom as the consistent exception. Together, these results support the system-level claim that action-label-free pretraining can improve downstream planning while using only a fraction of the trajectories with action annotations.

Coupled action and dynamics learning. The reference Full model obtains the highest mean long-horizon success among the component-ablation variants in Table 2. The independent pretraining-seed study further preserves its advantage over No-VQ and No-LAM for every paired seed and over Detached for two of three seeds. These results support the complete end-to-end objective: JEPA transition shaping directly organizes inferred actions around state changes, while the joint gradient path lets the action representation and dynamics predictor co-adapt. The prediction diagnostic clarifies why this coupling matters. An unconstrained continuous IDM can explain observed transitions with much lower prediction error while producing a substantially worse raw-action planner, because the training-time IDM sees the future state that is unavailable when the planner evaluates a proposed action.

Planning efficiency. Under the matched native workload, LaFreeWM completes a solve in 0.287 s compared with 0.700 s for Garrido et al. (Noisy), supporting practical use inside iterative planning. The unified-adapter control finds essentially equal core latency for LaFreeWM and LeWM; Appendix C.4 therefore reports native end-to-end timing and core-predictor latency separately.

Limitations. Our evidence is limited to four clean simulation benchmarks, leaving camera motion, embodiment changes, background dynamics, broader behavior distributions, and real-video scale for future evaluation. Raw-action planning still requires a small labeled subset for grounding. The Garrido et al. (Noisy) results use our common-interface simulation adaptation, and native latency reflects each system’s implementation and data movement alongside model computation.

7 Conclusion

We introduced LaFreeWM, a JEPA world model that learns dynamics without action labels by conditioning on discrete actions inferred from observation transitions, then uses a small labeled subset to ground the predictor for raw-action planning. Across four continuous-control tasks, this pretraining improves over label-only LeWM at matched 1% and 10% label budgets while retaining practical planning speed. The prediction–planning diagnostic identifies the central challenge: lower latent prediction error can coincide with substantially worse control when the inferred action carries transition detail that cannot serve as an executable action. These results motivate scaling observation-inferred action world models to more diverse video and real-robot settings.

Acknowledgments and Disclosure of Funding

Acknowledgments omitted for anonymous submission.

References

- Ha, D. and Schmidhuber, J. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. In *ICLR*, 2020.
- Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- LeCun, Y. A path towards autonomous machine intelligence. *OpenReview*, 2022. Version 0.9.2.
- Maes, L., Le Lidec, Q., Scieur, D., LeCun, Y., and Balestrieri, R. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- Zhou, G., Pan, H., LeCun, Y., and Pinto, L. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.
- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.
- Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., and Ballas, N. Self-supervised learning from images with a joint-embedding predictive architecture. In *ICCV*, 2023.
- Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., and Ballas, N. V-JEPA: Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.
- Sobal, V., Zhang, W., Cho, K., Balestrieri, R., Rudner, T. G. J., and LeCun, Y. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025.
- Pathak, D., Agrawal, P., Efros, A. A., and Darrell, T. Curiosity-driven exploration by self-supervised prediction. In *ICML*, 2017.
- Agrawal, P., Nair, A. V., Abbeel, P., Malik, J., and Levine, S. Learning to poke by poking: Experiential learning of intuitive physics. In *NeurIPS*, 2016.
- Bruce, J., Dennis, M., Edwards, A., Parker-Holder, J., Shi, Y., Hughes, E., Lai, M., Mavalankar, A., Steigerwald, R., Apps, C., et al. Genie: Generative interactive environments. In *ICML*, 2024.
- Ye, S., Jang, J., Jeon, B., Joo, S., Yang, J., Peng, B., Mandlekar, A., Tan, R., Chao, Y.-W., Lin, B. Y., et al. Latent action pretraining from videos. *arXiv preprint arXiv:2410.11758*, 2024.
- Tharwat, B., Nasser, Y., Abouzeid, A., and Reid, I. Latent action pretraining through world modeling. *arXiv preprint arXiv:2509.18428*, 2025.
- Liang, A., Czempin, P., Hong, M., Zhou, Y., Biyik, E., and Tu, S. CLAM: Continuous latent action models for robot learning from unlabeled demonstrations. *arXiv preprint arXiv:2505.04999*, 2025.
- Lee, J. M., Lee, D., Ju, S., Cho, T., Koo, J. W., Zhao, L., Hong, S., and Lee, J. MVP-LAM: Learning action-centric latent action via cross-viewpoint reconstruction. *arXiv preprint arXiv:2602.03668*, 2026.
- Zhang, C., Wang, J., Gao, Z., Su, Y., Dai, T., Zhou, C., Lu, J., and Tang, Y. CLAP: Contrastive latent action pretraining for learning vision-language-action models from human videos. *arXiv preprint arXiv:2601.04061*, 2026a.
- Gao, S., Liang, W., Zheng, K., Malik, A., Ye, S., Yu, S., Tseng, W.-C., Dong, Y., Mo, K., Lin, C.-H., et al. DreamDojo: A generalist robot world model from large-scale human videos. *arXiv preprint arXiv:2602.06949*, 2026.
- Jang, J., Ye, S., Lin, Z., Xiang, J., Bjorck, J., Fang, Y., Hu, F., Huang, S., Kundalia, K., Lin, Y.-C., et al. Dream-Gen: Unlocking generalization in robot learning through neural trajectories. *arXiv preprint arXiv:2505.12705*, 2025.
- Ye, S., Ge, Y., Zheng, K., Gao, S., Yu, S., Kurian, G., Indupuru, S., Tan, Y. L., Zhu, C., Xiang, J., et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- Garrido, Q., Nagarajan, T., Terver, B., Ballas, N., LeCun, Y., and Rabbat, M. Learning latent action world models in the wild. *arXiv preprint arXiv:2601.05230*, 2026.

- Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., Muckley, M., Rizvi, A., Roberts, C., Sinha, K., et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- Sun, J., Zhang, W., Qi, Z., Ren, S., Liu, Z., Zhu, H., Sun, G., Jin, X., and Chen, Z. VLA-JEPA: Enhancing vision-language-action models with latent world models. *arXiv preprint arXiv:2602.10098*, 2026.
- Zhang, Z., Wang, Y., Guan, Z., Yang, Y., Shi, B., Zong, T., Yi, H., Chao, G., Chen, X., Yang, T., et al. Delta-JEPA: Learning action-sensitive world models via latent difference decoding. *arXiv preprint arXiv:2606.31232*, 2026b.
- Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., Doersch, C., Pires, B. A., Guo, Z. D., Azar, M. G., et al. Bootstrap your own latent: A new approach to self-supervised learning. In *NeurIPS*, 2020.
- Kingma, D. P. and Welling, M. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- van den Oord, A., Vinyals, O., and Kavukcuoglu, K. Neural discrete representation learning. In *NeurIPS*, 2017.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. In *RSS*, 2023.
- Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., de Las Casas, D., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., Lillicrap, T., and Riedmiller, M. DeepMind Control Suite. *arXiv preprint arXiv:1801.00690*, 2018.
- Park, S., Frans, K., Eysenbach, B., and Levine, S. OGBench: Benchmarking offline goal-conditioned RL. *arXiv preprint arXiv:2410.20092*, 2024.
- Rubinstein, R. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.
- Bardes, A., Ponce, J., and LeCun, Y. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022.
- Balestriero, R. and LeCun, Y. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.

A Architecture details

A.1 Backbone configuration

LaFreeWM uses a ViT-Tiny backbone with the following configuration:

Parameter	Value
Patch size	14
Embedding dimension	192
Depth (layers)	12
Attention heads	3
Predictor depth	6
Predictor heads	16
Predictor MLP dimension	2048
Predictor dropout	0.1
Action latent dimension	24
History size (PushT, Cube)	3
History size (TwoRoom, Reacher)	1

Loss term	Weight	Notes
Next-CLS prediction MSE	1.0 (implicit)	Always active
SIGReg (encoder CLS)	0.09	Always active
JEPA transition loss	0.25	IDM active
LAM decoder loss	1.0	IDM active
VQ commitment loss	0.25	IDM active

A.2 Loss weight summary

A.3 Training and configuration details

All LaFreeWM variants are trained on a single NVIDIA H200 GPU with learning rate 5×10^{-5} , weight decay 10^{-3} , batch size 128, and SIGReg weight 0.09. The component ablations use a VQ codebook size of 64, commitment weight $\lambda_{\text{VQ}} = 0.25$, JEPA weight 0.25, and LAM weight 1.0. The action-label-free world model and fully supervised single-step baselines train for two epochs. Label-only LeWM trains for 10 epochs on the smaller labeled subset, so the comparison is matched in action supervision rather than optimization steps. Table 1 uses the selected per-environment configurations; Appendix C.2 reports the complete codebook sweep, and Table 3 gives the corresponding multi-horizon results.

A.4 VQ bottleneck details

The reported Full model updates its codebook entries with EMA. For mini-batch assignments k_i , EMA cluster counts N_j and latent sums $\mathbf{m}_j$ are updated as

$$N_j \leftarrow \gamma N_j + (1 - \gamma) \sum_i \mathbf{1}[k_i = j], \quad \mathbf{m}_j \leftarrow \gamma \mathbf{m}_j + (1 - \gamma) \sum_i \mathbf{1}[k_i = j] \mathbf{a}_i^{(\text{raw})}, \quad (9)$$

with $\mathbf{e}_j \leftarrow \mathbf{m}_j / N_j$ (using the implementation’s numerical smoothing). The encoder receives the commitment loss

$$\mathcal{L}_{\text{commit}} = \|\mathbf{a}^{(\text{raw})} - \text{sg}[\mathbf{e}_k]\|_2^2, \quad (10)$$

and the straight-through construction in Eq. 2 carries downstream gradients to the IDM encoder. The Full implementation uses EMA updates, while the Gradient-updated VQ row in Table 2 evaluates direct codebook optimization.

A.5 Action fine-tuning details

After label-free pre-training, the world model is fine-tuned on a small labeled subset to replace the VQ-bottlenecked IDM pathway with direct raw-action conditioning. The matched component study uses the same labeled manifest, staged fine-tuning cadence, optimizer, batch cap, and raw-action conditioner across all variants. After fine-tuning, CEM plans directly with raw actions through the grounded conditioner.

B Evaluation configuration details

All reported single-step experiments use a dataset frameskip of 1 and 96×96 RGB observations. PushT and Cube use a history of three observations, while TwoRoom and Reacher use a history of one. Goal horizons $h \in \{1, 5, 10, 15\}$ use evaluation budgets of $\{5, 10, 20, 30\}$ primitive steps, respectively. Each evaluation contains 50 episodes; the $h = 15$ results aggregate seeds 0, 1, and 2, while the shorter horizons use seed 0. The same task trajectories, goal construction, planner settings, and evaluation implementation are used across the controlled comparisons.

C Additional results

C.1 Multi-horizon results across environments

Table 3 reports success rates at goal horizons $h \in \{1, 5, 10, 15\}$ across all four environments under the single-step regime. The methods and configurations match Table 1; the $h=15$ values reproduce that comparison, while $h<15$ values are seed-0 point estimates.

Table 3: Multi-horizon planning success (% , $\uparrow$) under frame-skip 1. $h = 15$: mean $\pm$ population std over three evaluation seeds; $h < 15$: seed-0 point estimates.

Env	Method	Labels	h=1	h=5	h=10	h=15
PushT	LeWM	Full	88	62	50	30.0 $\pm$ 10.7
	DINO-WM	Full	100	100	100	86.7 $\pm$ 3.8
	PLDM	Full	98	60	28	20.7 $\pm$ 4.1
	LeWM	1%	26	30	20	6.7 $\pm$ 0.9
	Garrido et al. (Noisy)	1%	54	38	16	13.3 $\pm$ 3.4
	LaFreeWM	1%	96	60	36	33.3 $\pm$ 1.9
	LeWM	10%	86	64	42	26.0 $\pm$ 5.7
	Garrido et al. (Noisy)	10%	74	50	32	16.0 $\pm$ 8.2
	LaFreeWM	10%	100	96	78	60.7 $\pm$ 3.4
	LeWM	Full	100	100	100	92.7 $\pm$ 2.5
	DINO-WM	Full	100	100	100	100.0 $\pm$ 0.0
	PLDM	Full	100	94	74	68.7 $\pm$ 9.0
TwoRoom	LeWM	1%	100	96	64	50.7 $\pm$ 5.2
	Garrido et al. (Noisy)	1%	100	98	94	91.3 $\pm$ 5.7
	LaFreeWM	1%	100	100	98	76.7 $\pm$ 10.4
	LeWM	10%	100	98	94	78.0 $\pm$ 4.9
	Garrido et al. (Noisy)	10%	100	100	96	92.7 $\pm$ 5.0
	LaFreeWM	10%	100	100	92	83.3 $\pm$ 9.6
	LeWM	Full	80	34	56	62.7 $\pm$ 8.2
	DINO-WM	Full	98	62	64	61.3 $\pm$ 6.2
	PLDM	Full	66	24	26	24.0 $\pm$ 10.7
	LeWM	1%	54	22	26	20.0 $\pm$ 4.3
	Garrido et al. (Noisy)	1%	26	26	18	6.7 $\pm$ 5.2
	LaFreeWM	1%	64	64	76	67.3 $\pm$ 7.4
Reacher	LeWM	10%	76	40	46	60.7 $\pm$ 6.2
	Garrido et al. (Noisy)	10%	62	44	74	61.3 $\pm$ 1.9
	LaFreeWM	10%	68	68	82	70.7 $\pm$ 5.7
	LeWM	Full	100	100	88	77.3 $\pm$ 3.4
	DINO-WM	Full	100	96	88	74.7 $\pm$ 6.2
	PLDM	Full	100	94	76	68.7 $\pm$ 2.5
	LeWM	1%	100	80	76	63.3 $\pm$ 7.7
	Garrido et al. (Noisy)	1%	100	94	94	60.0 $\pm$ 9.9
	LaFreeWM	1%	100	98	86	70.7 $\pm$ 3.4
	LeWM	10%	100	96	84	76.7 $\pm$ 2.5
	Garrido et al. (Noisy)	10%	100	92	94	71.3 $\pm$ 2.5
	LaFreeWM	10%	100	100	94	82.0 $\pm$ 7.1

C.2 Codebook sensitivity

Table 4 reports the full 10%-label sweep at $h = 15$. $K = 16$ has the highest average across the four tasks, but sensitivity is non-monotonic and task-dependent: $K = 8$ is highest on Reacher and $K = 128$ on Cube.

C.3 Conditional pretraining-seed stability

We repeat Full, Detached, No-LAM, and No-VQ over three independent pretraining seeds. Each pretrained model uses the same fine-tuning seed and is evaluated with three planner/environment seeds, isolating variation due to pretraining.

Table 4: Full-configuration codebook sensitivity at $h = 15$ (10% labels), mean $\pm$ population std over three evaluation seeds.

Codebook	PushT	TwoRoom	Reacher	Cube
$K = 8$	48.7 ± 5.2	78.0 ± 7.5	70.7 ± 5.7	76.7 ± 5.2
$K = 16$	60.7 ± 3.4	83.3 ± 9.6	66.7 ± 9.0	79.3 ± 6.6
$K = 64$	59.3 ± 8.2	75.3 ± 12.3	64.7 ± 4.7	76.0 ± 5.9
$K = 128$	51.3 ± 3.4	76.7 ± 9.8	64.7 ± 2.5	82.0 ± 7.1

Table 5: PushT $h = 15$ success (%) across three pretraining seeds, with fixed fine-tuning seed. Full beats No-VQ and No-LAM for all paired seeds and exceeds Detached for two of three.

Variant	Success across pretraining seeds
Full	54.4 ± 8.1
Detached prediction gradient	50.9 ± 2.3
Without LAM	29.8 ± 1.9
Without VQ	15.8 ± 2.8

C.4 Native-planner efficiency

Table 6 uses the same PushT CEM workload on H200: horizon 5, 300 candidates, 30 iterations, and top- k 30. The synchronized timer encloses one complete solve and excludes loading, simulator interaction, rendering, logging, and success evaluation.

Table 6: End-to-end native-planner measurements on H200. Latency is mean $\pm$ population std over three per-process medians. Each row uses the method’s native precision, caching, and input path; the unified-adaptor control below compares core-planner latency.

Planner	Precision	Params.	Latency (s)	Peak GiB	Relative
LaFreeWM	BF16	27.52M	0.287 ± 0.008	0.43	$1.00 \times$
Garrido et al. (Noisy)	BF16	376.75M	0.700 ± 0.012	0.85	$2.44 \times$
LeWM	FP32	18.03M	2.885 ± 0.086	0.46	$10.06 \times$
PLDM	FP32	18.03M	2.996 ± 0.017	0.46	$10.44 \times$
DINO-WM	FP32	52.32M	42.627 ± 0.164	17.82	$148.60 \times$
DINO-WM cached control	FP16	52.32M	20.126 ± 0.005	15.83	$70.16 \times$

A separate unified-adaptor control measured essentially equal LaFreeWM and LeWM core-planner latency (189.46 versus 186.85 ms on RTX 5090), complementing the native end-to-end measurements with a controlled predictor comparison.

D Broader impact

This work contributes to world models for robotic planning. Self-supervised world models could reduce dependence on expensive labeled demonstrations, lowering barriers to deploying robots in new environments. The VQ-bottlenecked IDM design could improve scalability by enabling world-model training on large video datasets without action annotations.

World models that learn from video without action supervision could be applied to settings where autonomous behavior is undesirable or where training data encodes unsafe behaviors. Extending the current 2D and simulated 3D results to real-world deployment requires further validation. The fine-tuning step on labeled data introduces a natural supervision checkpoint: the need for some labels at deployment provides an opportunity for human oversight.

E Reproducibility

The implementation release will include the model, training, grounding, planning, and evaluation code together with the exact experiment configurations used in the paper. These configurations record the data split and label manifest, frameskip and history settings, optimization schedule, checkpoint, planner parameters, evaluation budgets, and seeds for each reported result. Code and experiment configurations will be made publicly available with the final paper.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state the contribution as a JEPA world model with observation-inferred discrete actions for action-label-free pretraining, followed by limited-label raw-action grounding. They also summarize the label-matched comparison, fully supervised references, planning efficiency, and prediction–planning diagnostic at the scope evaluated in the paper.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 6 covers clean-simulation scope, limited-label raw-action grounding, broader behavior and real-video evaluation, the controlled noisy-continuous adaptation, and the implementation dependence of native timing.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results or proofs. All contributions are architectural and empirical.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Full architecture details, hyperparameters, loss weights, training budgets, and evaluation configurations are provided in Section 3 and Appendices A–C. The main IDM variant, label-matched and latent-action baselines, matched ablation interface, and evaluation protocol are specified.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Code and exact experiment configurations will be made publicly available with the final paper. The benchmark datasets are publicly available, while the implementation is not yet publicly accessible.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4 specifies the environments, label budgets, evaluation seeds and episode counts, horizon-dependent budgets, baselines, and planner interface. Appendices A–C provide optimization budgets, hardware, architecture, evaluation configuration, sensitivity, stability, and native-planner details.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The primary comparison and matched component ablation report $h=15$ mean $\pm$ population standard deviation over three planner/environment evaluation seeds with 50 episodes per seed. Shorter-horizon cells are seed-0 point estimates. A separate three-pretraining-seed study fixes the fine-tuning seed; the paper explicitly distinguishes this conditional variation from a fully crossed training-seed analysis.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The paper now states that all reported experiments were run on a single NVIDIA H200 GPU, but it does not yet report memory requirements, per-run wall-clock time, or total project compute. We therefore answer [No] for now and will update this item if those details are added.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research involves standard machine learning methods on publicly available simulated data (PushT). No human subjects, private data, or ethically sensitive applications are involved.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Appendix D discusses positive impacts (reducing dependence on action annotations and scaling world-model training to larger unlabeled interaction datasets), negative impacts (potential misuse of video-trained world models), and deployment-time supervision as a natural checkpoint for human oversight.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper presents a world model for simple 2D manipulation tasks. The model and data do not pose risks of misuse comparable to language models or image generators.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: The PushT environment and dataset are cited [Chi et al., 2023]. The LeWorld-Model codebase that this work extends is cited [Maes et al., 2026]; DINO-WM and the closest noisy-continuous latent-action world model are also cited [Zhou et al., 2024, Garrido et al., 2026].

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[N/A\]](#)

Justification: No new datasets or pretrained models are released. The codebase will be released upon acceptance with documentation.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human subjects research.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not used as a component of the core method. The method is a JEPA-style world model with no LLM components.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.